Logo de l'école
figures/logos/logo-ecole.png

République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
[Université de rattachement]
EPI Sousse

Logo de l'entreprise
figures/logos/logo-entreprise.png

Rapport de stage

Cycle ingénieur — Ingénierie Cloud

SentinelOps

Plateforme auto-hébergée de supervision et d'alerte intelligente pour infrastructures conteneurisées

Réalisé par

Mohamed Amine Benzid

Organisme d'accueil : Barmaj Information Technology

Encadrant professionnel : M. Yousef Abed Elberi

Encadrant académique : [Nom de l'encadrant académique]

Période du stage : [du JJ/MM/AAAA au JJ/MM/AAAA]

Année universitaire 2026 – 2027

Dédicace

À mes chers parents, pour leurs sacrifices, leur soutien sans faille et leur confiance tout au long de mes études. Ce travail est le fruit de vos encouragements.

À ma famille et à mes amis, pour leur présence et leurs encouragements dans les moments de doute.

À tous ceux qui m'ont appris quelque chose en chemin.

Mohamed Amine Benzid

Remerciements

Au terme de ce stage, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Je remercie chaleureusement **M. Yousef Abed Elberi**, mon encadrant professionnel au sein de **Barmaj Information Technology**, pour sa disponibilité, ses conseils techniques et la confiance qu'il m'a accordée tout au long de ce projet.

J'adresse également mes sincères remerciements à **[Nom de l'encadrant académique]**, mon encadrant académique à EPI Sousse, pour son suivi, ses remarques pertinentes et son accompagnement dans la rédaction de ce rapport.

Mes remerciements vont aussi à l'ensemble de l'équipe de Barmaj Information Technology pour son accueil et pour l'environnement de travail stimulant qu'elle m'a offert.

Enfin, je remercie les membres du jury d'avoir accepté d'évaluer ce travail.

Résumé

Ce rapport présente le travail réalisé au sein de Barmaj Information Technology dans le cadre de mon stage : la conception et la réalisation de **SentinelOps**, une plateforme auto-hébergée de supervision et d'alerte pour serveurs et conteneurs Docker. L'objectif est de remplacer l'assemblage habituel de plusieurs outils (Prometheus, Alertmanager, Grafana) par une solution qui se déploie en une seule commande et qui produit des alertes expliquées.

La plateforme collecte toutes les cinq secondes les métriques de l'hôte, des conteneurs et de tout exportateur Prometheus. Elle détecte les incidents par des règles de seuil et par un détecteur statistique d'anomalies (score z robuste fondé sur la médiane et l'écart absolu médian). Un automate d'états (en attente, déclenchée, résolue) élimine les pics transitoires, regroupe les alertes en incidents et les résout automatiquement. Les notifications Slack et e-mail sont fiabilisées par le patron « transactional outbox ». La solution applique une sécurité par défaut (API fermée par défaut, sessions hachées et expirables, accès Docker filtré par des proxies, conteneurs non privilégiés) et s'appuie sur une chaîne d'intégration continue avec 51 tests automatisés et l'analyse de vulnérabilités.

Mots-clés : supervision, alerting, détection d'anomalies, Docker, DevSecOps, intégration continue, observabilité.

Abstract

This report presents the work carried out at Barmaj Information Technology during my internship: the design and implementation of **SentinelOps**, a self-hosted monitoring and alerting platform for servers and Docker containers. Its goal is to replace the usual combination of several tools (Prometheus, Alertmanager, Grafana) with a solution deployed in a single command that produces explained alerts.

The platform collects host, container and Prometheus exporter metrics every five seconds. It detects incidents with threshold rules and a statistical anomaly detector (a robust z-score based on the median and the median absolute deviation). An alert state machine (pending, firing, resolved) filters transient spikes, groups alerts into incidents

and resolves them automatically. Slack and e-mail notifications are made reliable with the transactional outbox pattern. The solution is secure by default (deny-by-default API, hashed and expiring sessions, Docker access filtered by proxies, unprivileged containers) and relies on a continuous integration pipeline with 51 automated tests and vulnerability scanning.

Keywords: monitoring, alerting, anomaly detection, Docker, DevSecOps, continuous integration, observability.

Table des matières

Liste des abréviations **xiii**

Introduction générale **1**

1 Contexte général du projet **3**

1.1 Présentation de l'organisme d'accueil 3

1.1.1 Barmaj Information Technology 3

1.1.2 Activités et services 3

1.1.3 Organisation 3

1.2 Cadre du projet 4

1.3 Problématique 4

1.4 Étude de l'existant 5

1.4.1 Notions de base 5

1.4.2 Solutions existantes 5

1.4.3 Synthèse comparative 6

1.4.4 Critique de l'existant 6

1.5 Solution proposée 7

1.6 Méthodologie de travail 7

1.7 Planning 8

2 Analyse et spécification des besoins **10**

2.1 Identification des acteurs 10

2.2 Besoins fonctionnels 10

2.3 Besoins non fonctionnels 12

2.4 Diagramme de cas d'utilisation global 12

2.5 Description des cas d'utilisation principaux 13

3 Conception **15**

3.1 Architecture globale 15

3.2 Conception de l'agent 17

3.2.1 Chaîne de traitement 17

3.2.2 Diagramme de classes 18

3.2.3 Automate d'états d'une alerte 19

3.2.4 Corrélation en incidents 20

3.3 Détection d'anomalies 21

3.3.1 Principe : le score z robuste 21

3.3.2 Exemple de calcul 21

3.3.3 Pourquoi la médiane plutôt que la moyenne 21

3.4 Modèle de données 22

3.4.1 Migrations versionnées 23

3.4.2 Outbox transactionnelle 23

3.5 Diagrammes de séquence 24

3.5.1 Cycle de collecte et de détection 24

3.5.2 Livraison des notifications 24

3.5.3 Authentification et flux temps réel 25

3.5.4 Acquittement et résolution manuelle 25

4 Réalisation **26**

4.1 Environnement de travail 26

4.1.1 Environnement matériel 26

4.1.2 Environnement logiciel 27

4.2 Choix technologiques 28

4.3 Organisation du code 28

4.4 Extraits de code significatifs 29

4.4.1 Cycle de vie des alertes 29

4.4.2 Score z robuste 29

4.4.3 Migrations sous verrou exclusif 30

4.4.4 API fermée par défaut 30

4.4.5 Livraison des notifications 30
4.5 Lanceur en une commande 31
4.6 Interfaces de l'application 32
4.6.1 Authentification 32
4.6.2 Vue d'ensemble 33
4.6.3 Détail d'un incident 34
4.6.4 Courbe d'anomalie 35
4.6.5 Règles, paramètres et journaux 35
4.6.6 Thème clair et affichage mobile 36
4.6.7 Notifications Slack 37

5 Sécurité, tests, intégration continue et déploiement 38
5.1 Sécurité de l'application 39
5.2 Sécurité des conteneurs 39
5.3 Intégration continue 41
5.4 Tests 43
5.4.1 Tests automatisés 43
5.4.2 Scénarios de validation de bout en bout 44
5.5 Déploiement sur un serveur 44
5.6 Évaluation 47
5.6.1 Comparaison des deux estimateurs 47
5.6.2 Délais de détection et de notification 47
5.6.3 Consommation de ressources 48
5.6.4 Limites 48

Conclusion générale et perspectives 49

Bibliographie et webographie 51

A Guide d'installation 53

B Référence de l'API REST 55

C Règles d'alerte par défaut **57**

D Variables d'environnement **58**

E Liste des captures d'écran **60**

Table des figures

1.1 Organigramme de Barmaj Information Technology 4
1.2 Diagramme de Gantt du stage [à ajuster aux dates réelles] 9

2.1 Diagramme de cas d'utilisation global 13

3.1 Architecture globale de SentinelOps 15
3.2 Chaîne de traitement de l'agent 18
3.3 Diagramme de classes de l'agent 19
3.4 Automate d'états d'une alerte 20
3.5 Modèle de données (principales tables) 22
3.6 Diagramme de séquence du cycle de détection 24
3.7 Diagramme de séquence de la livraison des notifications 24
3.8 Diagramme de séquence de l'authentification 25
3.9 Diagramme de séquence de l'acquittement et de la résolution 25

4.1 Sortie de `sen run` jusqu'au message « Ready » 32
4.2 Sortie de `sen status` 32
4.3 Authentification 33
4.4 Vue d'ensemble : système en état normal 33
4.5 Vue d'ensemble pendant un incident 33
4.6 Détail d'un incident : onglet « Pourquoi » (preuves et valeurs observées) 34
4.7 Détail d'un incident : chronologie 34
4.8 Détail d'un incident : instantané des processus 34
4.9 Détail d'un incident : brouillon de post-mortem 35
4.10 Courbe d'une métrique avec sa bande attendue et le point anormal . . 35
4.11 Gestion des règles d'alerte 35
4.12 Paramètres : canaux de notification et heures calmes 36

4.13 Journaux d'un conteneur et bouton de redémarrage 36
4.14 Thème clair et affichage adaptatif 36
4.15 Messages reçus sur Slack 37

5.1 Les conteneurs de SentinelOps en fonctionnement (`docker ps`) 40
5.2 Vérification des contrôles d'accès 40
5.3 Pipeline d'intégration continue 41
5.4 Pipeline GitHub Actions réussi 41
5.5 Analyse Trivy bloquant une image vulnérable 42
5.6 Rapport `npm audit` 42
5.7 Demandes de mise à jour ouvertes par Dependabot 42
5.8 Exécution des tests 43
5.9 Diagramme de déploiement 45
5.10 Instance cloud utilisée pour le test 46
5.11 Règles du pare-feu (groupe de sécurité) de l'instance 46
5.12 Dashboard supervisant le serveur distant 46

Liste des tableaux

1.1 Comparaison des solutions de supervision étudiées 6
1.2 Feuille de route versionnée du produit 8

2.1 Acteurs du système 10
2.2 Besoins fonctionnels 12
2.3 Besoins non fonctionnels 12
2.4 Description du cas « S'authentifier » 13
2.5 Description du cas « Traiter un incident » 14
2.6 Description du cas « Gérer une règle » 14

3.1 Rôle des conteneurs 16
3.2 Règles de conception de la version 0.2 17
3.3 Propriété et conservation des tables 23

4.1 Environnement matériel 26
4.2 Environnement logiciel 27
4.3 Choix technologiques 28
4.4 Commandes du lanceur `sen` 31

5.1 Mesures de sécurité de l'application 39
5.2 Proxies d'accès à Docker 40
5.3 Répartition des tests automatisés 43
5.4 Scénarios de validation manuelle 44
5.5 Comportement face à un saut durable 47
5.6 Délais de détection et de notification 47
5.7 Consommation de ressources (relevée avec `docker stats` et `docker images`) 48

B.1 Routes de l'API REST 55

C.1 Règles livrées par défaut 57

D.1 Variables d'environnement 58

E.1 Captures d'écran à fournir 60

Liste des abréviations

API Application Programming Interface

CI/CD Continuous Integration / Continuous Delivery

CPU Central Processing Unit

CSP Content Security Policy

CVE Common Vulnerabilities and Exposures

DevSecOps Development, Security and Operations

HTTP HyperText Transfer Protocol

IaC Infrastructure as Code

JSON JavaScript Object Notation

MAD Median Absolute Deviation (écart absolu médian)

MTTD Mean Time To Detect

MTTR Mean Time To Resolve

OWASP Open Worldwide Application Security Project

PBKDF2 Password-Based Key Derivation Function 2

RAM Random Access Memory

REST Representational State Transfer

SBOM Software Bill of Materials

SMTP Simple Mail Transfer Protocol

SQL Structured Query Language

SRE Site Reliability Engineering

SSH Secure Shell

SSRF Server-Side Request Forgery

UI User Interface

UML Unified Modeling Language

VPS Virtual Private Server

WAL Write-Ahead Logging

XSS Cross-Site Scripting

Introduction générale

Les applications modernes reposent sur des serveurs, des conteneurs et des services qui doivent fonctionner en continu. Lorsqu'un processeur sature, qu'un conteneur manque de mémoire ou qu'un disque se remplit, l'équipe doit le savoir rapidement, comprendre la cause et agir avant que les utilisateurs ne soient touchés. C'est le rôle de la **supervision** (en anglais *monitoring*) et de l'**alerte**.

Les outils de référence existent : Prometheus collecte les métriques, Alertmanager envoie les alertes et Grafana les affiche. Ils sont puissants, mais ils forment trois produits distincts à installer, configurer et relier entre eux. Pour une petite équipe, une startup ou un projet étudiant, cette mise en place prend plusieurs jours. J'en ai fait l'expérience lors de mon projet de fin d'année : la partie supervision a été la plus longue et la plus fragile à configurer. De plus, ces outils déclenchent des alertes sur des seuils fixes, sans expliquer pourquoi une alerte s'est déclenchée ni regrouper les alertes liées.

C'est dans ce contexte que s'inscrit mon stage au sein de **Barmaj Information Technology**. Le projet, nommé **SentinelOps**, a pour objectif de concevoir et réaliser une plateforme de supervision et d'alerte **auto-hébergée**, déployable en **une seule commande**, qui collecte les métriques des serveurs et des conteneurs, détecte les incidents par des règles et par une analyse statistique, explique chaque alerte avec des preuves mesurées et notifie les équipes de manière fiable, le tout avec un niveau de sécurité adapté à un environnement de production.

Ce rapport est organisé en cinq chapitres :

- le **chapitre 1** présente l'organisme d'accueil, le contexte du projet, l'étude des solutions existantes et la méthodologie de travail ;
- le **chapitre 2** spécifie les besoins fonctionnels et non fonctionnels, ainsi que les acteurs et les cas d'utilisation ;
- le **chapitre 3** décrit la conception : architecture, modèle de données, automate d'états des alertes, détection d'anomalies et principales interactions ;
- le **chapitre 4** présente la réalisation : environnement, choix technologiques, implémentation des modules clés et interfaces ;
- le **chapitre 5** traite de la sécurité, de la conteneurisation, de la chaîne d'intégration continue, des tests, du déploiement et de l'évaluation.

Une conclusion générale dresse le bilan du travail et présente les perspectives.

CHAPITRE 1

Contexte général du projet

Introduction

Ce chapitre situe le projet. Il présente d'abord l'organisme d'accueil, puis le cadre et la problématique du stage. Il propose ensuite une étude des solutions de supervision existantes, dont les limites motivent la solution proposée, et se termine par la méthodologie de travail et le planning suivis.

1.1 Présentation de l'organisme d'accueil

1.1.1 Barmaj Information Technology

Ce stage s'est déroulé au sein de **Barmaj Information Technology**, [**présentation de l'entreprise : date de création, siège, secteur d'activité, taille de l'équipe, principaux clients ou produits (3 à 5 lignes)**].

1.1.2 Activités et services

[**Décrire les principaux domaines d'activité de l'entreprise, par exemple : développement de solutions logicielles, services cloud, intégration, conseil. Une liste à puces de 3 à 6 éléments suffit.**]

1.1.3 Organisation

[**Présenter la structure de l'entreprise et situer l'équipe ou le département dans lequel le stage a été effectué, sous la responsabilité de M. Yousef Abed Elberi.**]

Capture à insérer

`figures/captures/cap-01-organigramme.png`

Organigramme de Barmaj Information Technology

Fig. 1.1 : Organigramme de Barmaj Information Technology

1.2 Cadre du projet

Ce travail a été réalisé dans le cadre du stage du Cycle ingénieur — Ingénierie Cloud à EPI Sousse, au sein de Barmaj Information Technology, **[du JJ/MM/AAAA au JJ/MM/AAAA]**. Il porte sur la conception et la réalisation complète d'un produit logiciel, de l'étude des besoins jusqu'au déploiement, en suivant les pratiques DevOps et DevSecOps : conteneurisation, intégration continue, tests automatisés et sécurité intégrée dès la conception.

1.3 Problématique

Superviser une infrastructure consiste à répondre en permanence à trois questions : *quelque chose ne va-t-il pas ?*, *pourquoi ?* et *qui doit agir ?* Les équipes réduites (startups, petites entreprises, projets étudiants) rencontrent trois difficultés :

1. **La complexité de mise en place.** La pile de référence associe Prometheus (collecte), Alertmanager (routage des alertes) et Grafana (visualisation). Chaque outil a sa configuration, son format et son cycle de mise à jour ; les relier demande du temps et des compétences que ces équipes n'ont pas toujours.
2. **Le bruit des alertes.** Des alertes fondées uniquement sur des seuils fixes se déclenchent sur des pics passagers, se répètent pour un même problème et ne sont pas regroupées, ce qui provoque une « fatigue d'alerte » : les vraies alertes finissent par être ignorées [1].
3. **Le manque d'explication.** Une alerte indique qu'un seuil est dépassé, mais pas quel processus en est la cause, ni depuis quand la situation dure.

Problématique

Comment offrir à une petite équipe une supervision **installable en une commande**, qui produit **peu d'alertes mais des alertes fiables et expliquées**, tout en restant **sécurisée** et **maintenable** comme un produit professionnel ?

1.4 Étude de l'existant

1.4.1 Notions de base

- **Métrique** : valeur numérique mesurée à intervalle régulier (utilisation CPU, mémoire, nombre de requêtes). C'est la matière première de la supervision.
- **Règle d'alerte** : condition sur une métrique, par exemple `cpu > 85 %` pendant 60 secondes.
- **Incident** : ensemble d'alertes liées à une même cause.
- **Observabilité** : capacité à comprendre l'état interne d'un système à partir de ses métriques, de ses journaux et de ses traces. La supervision en est le premier niveau.

1.4.2 Solutions existantes

Prometheus, Alertmanager et Grafana. Prometheus collecte des séries temporelles en interrogeant des *exportateurs* et évalue des règles d'alerte écrites en PromQL, avec une clause `for` qui impose une durée avant le déclenchement [2]. Alertmanager déduplique, regroupe et route les alertes vers les canaux de notification [3]. Grafana assure la visualisation [4]. Cette pile est un standard de l'industrie, mais elle se compose de trois produits à configurer séparément et ne propose pas de détection d'anomalies statistique native.

Zabbix. Solution de supervision complète et ancienne, avec agent, serveur, base de données et interface web [5]. Très riche fonctionnellement, elle demande cependant une base de données externe et une configuration conséquente.

Netdata. Agent de supervision temps réel très léger, avec des tableaux de bord prêts à l'emploi et une détection d'anomalies intégrée [6]. Il est orienté vers la visualisation fine d'un serveur plutôt que vers la gestion du cycle de vie des alertes et des incidents.

Datadog. Plateforme d'observabilité en mode SaaS, très complète (métriques, journaux, traces, détection d'anomalies) [7]. Les données quittent l'infrastructure de l'entreprise et la facturation dépend du nombre d'hôtes, ce qui peut être coûteux pour une petite équipe.

1.4.3 Synthèse comparative

Critère	**Prometheus + Grafana**	**Zabbix**	**Netdata**	**Datadog**	**SentinelOps**
Auto-hébergé	oui	oui	oui	non (SaaS)	oui
Installation	3 outils à relier	serveur + base externe	simple	agent + compte	une commande
Règles avec durée	oui (`for`)	oui	oui	oui	oui
Détection d'anomalies	non native	fonctions de tendance	oui	oui	oui (score z robuste)
Alerte expliquée (preuves, processus)	non	partiel	partiel	oui	oui
Regroupement en incidents	via Alertmanager	oui	limité	oui	oui
Coût	gratuit	gratuit	gratuit / payant	par hôte	gratuit

Tab. 1.1 : Comparaison des solutions de supervision étudiées

1.4.4 Critique de l'existant

Les solutions open source sont puissantes mais demandent un effort d'intégration important, tandis que la solution SaaS la plus complète a un coût récurrent et externalise les données. Aucune ne réunit à la fois une installation en une commande, une gestion complète du cycle de vie des alertes, des alertes expliquées et une sécurité par défaut pour une petite équipe.

1.5 Solution proposée

SentinelOps est une plateforme auto-hébergée qui se lance avec la commande `sen run` et qui :

- **collecte** toutes les cinq secondes les métriques de l'hôte, des conteneurs Docker et de tout exportateur Prometheus ;
- **détecte** les incidents par des règles de seuil et par un détecteur statistique d'anomalies ;
- **décide** grâce à un automate d'états qui filtre les pics passagers, évite les doublons et regroupe les alertes en incidents ;
- **explique** chaque alerte avec les valeurs mesurées et les processus les plus consommateurs au moment du déclenchement ;
- **notifie** par Slack et e-mail avec garantie de livraison ;
- **affiche** l'ensemble en temps réel dans une interface web.

SentinelOps ne rejette pas l'écosystème existant : tout exportateur Prometheus peut être interrogé, ce qui permet de superviser des applications tierces sans développement supplémentaire.

1.6 Méthodologie de travail

Le projet a suivi une **approche itérative et incrémentale**, guidée par une feuille de route versionnée. Chaque version livre un produit utilisable, puis est revue avant d'ajouter de nouvelles fonctionnalités. Chaque itération suit le même cycle : analyse, conception, réalisation, tests et revue.

Version	Contenu
v0.1 — Core	collecte, règles, déduplication, corrélation, tableau de bord, Slack/e-mail, déploiement Docker Compose
v0.2 — Intelligence	couche de détection extensible, détection d'anomalies, gestion d'incidents, durcissement
v0.3 — Assistant IA	explication par un modèle de langage local (optionnel), suivi des changements
v1.0 — Plateforme	prévisions, coûts cloud, modules Terraform, remédiation sous validation humaine

Tab. 1.2 : Feuille de route versionnée du produit

Le stage couvre la version 0.1 complète et le début de la version 0.2. Une étape importante a été une **revue de code** complète de la première version, qui a identifié seize problèmes (fonctionnels, de sécurité et de qualité) ; leur correction a conduit à la refonte décrite dans les chapitres 3 à 5.

1.7 Planning

La figure 1.2 présente le déroulement prévisionnel du stage.

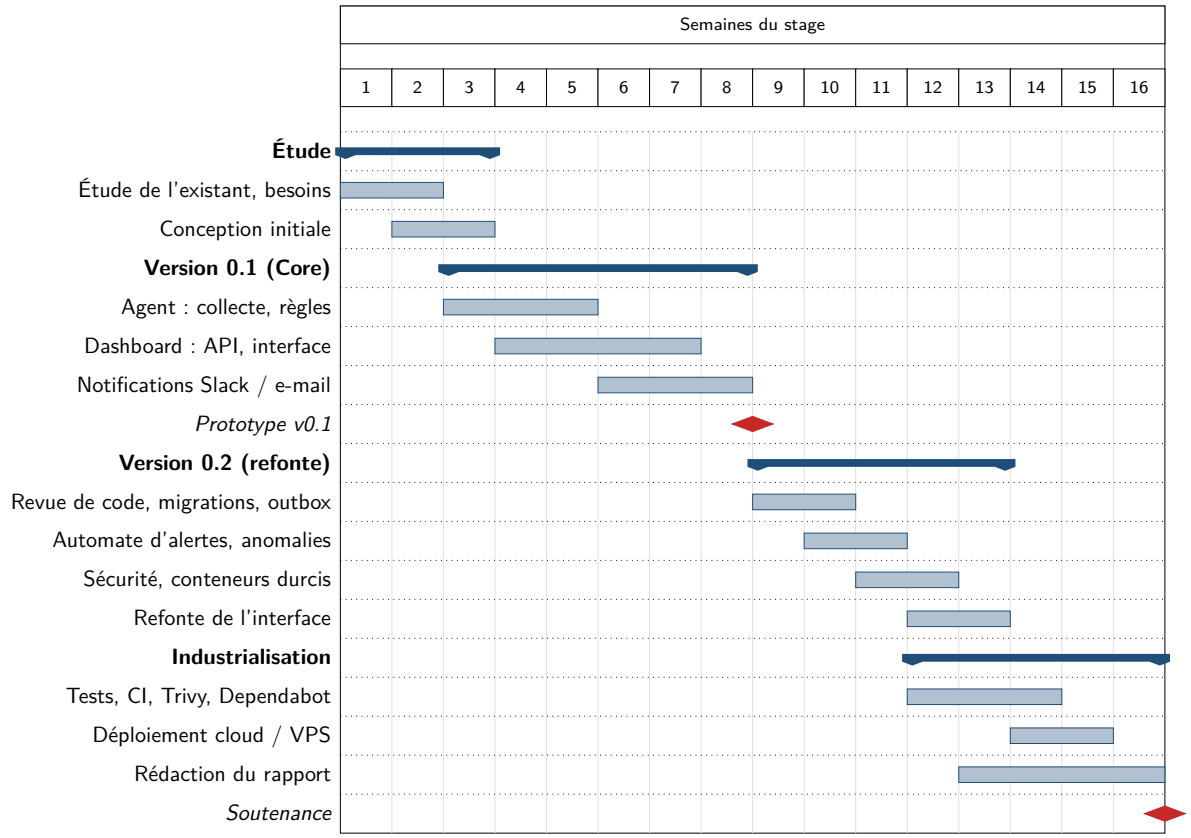


Fig. 1.2 : Diagramme de Gantt du stage **[à ajuster aux dates réelles]**

Conclusion

Ce chapitre a présenté le contexte du stage et montré, à travers l'étude de l'existant, le besoin d'une solution de supervision simple à déployer, aux alertes fiables et expliquées. Le chapitre suivant formalise ces besoins.

CHAPITRE 2

Analyse et spécification des besoins

Introduction

Ce chapitre traduit la problématique en exigences précises. Il identifie les acteurs du système, puis détaille les besoins fonctionnels et non fonctionnels, avant de les représenter par un diagramme de cas d'utilisation et la description des cas principaux.

2.1 Identification des acteurs

Acteur	Type	Rôle
Administrateur (opérateur)	humain	se connecte à l'interface, suit l'état de l'infrastructure, traite les incidents, gère les règles et les notifications
Agent de collecte	système	collecte les métriques, détecte les problèmes et fait évoluer les alertes ; fonctionne sans intervention humaine
Service de notification	système externe	Slack (webhook) et serveur SMTP qui reçoivent les messages d'alerte et de résolution

Tab. 2.1 : Acteurs du système

2.2 Besoins fonctionnels

Réf.	Besoin	Description
BF-01	Collecter les métriques	CPU, mémoire, disque et charge de l'hôte ; CPU, mémoire, réseau et redémarrages de chaque conteneur ; séries choisies de tout exportateur Prometheus ; toutes les 5 secondes.

Réf.	Besoin	Description
BF-02	Gérer des règles de seuil	créer, modifier, activer, désactiver et supprimer des règles (métrique, opérateur, seuil, durée, sévérité). Les règles par défaut sont fournies dans un fichier YAML.
BF-03	Détecter les anomalies	signaler une valeur anormale par rapport au comportement récent d'une métrique, sans seuil écrit à la main.
BF-04	Gérer le cycle de vie des alertes	n'ouvrir une alerte qu'après la durée définie, ne jamais la dupliquer, la résoudre automatiquement quand la condition disparaît.
BF-05	Regrouper en incidents	rattacher à un même incident les alertes d'une même entité apparues dans une fenêtre de temps.
BF-06	Expliquer les alertes	conserver les valeurs mesurées qui justifient l'alerte et une capture des processus les plus consommateurs.
BF-07	Notifier	envoyer un message Slack et un e-mail au déclenchement et à la résolution, avec nouvelles tentatives en cas d'échec et heures silencieuses.
BF-08	Traiter une alerte	acquitter, retirer l'acquittement, résoudre manuellement, rouvrir.
BF-09	Visualiser en temps réel	état de chaque entité, courbes avec plage attendue, incidents, mise à jour sans rechargement de la page.
BF-10	Consulter l'historique	métriques sur 15 min, 1 h, 6 h et 24 h ; chronologie et post-mortem d'un incident.
BF-11	Opérer les conteneurs	consulter les journaux d'un conteneur et le redémarrer depuis l'interface.
BF-12	Authentifier	connexion par identifiant et mot de passe, changement obligatoire du mot de passe initial, déconnexion.
BF-13	Configurer	paramétrer Slack, SMTP, heures silencieuses et détection d'anomalies depuis l'interface.

Réf.	Besoin	Description
BF-14	Déployer en une commande	démarrer l'ensemble et indiquer quand la plateforme est réellement prête.

Tab. 2.2 : Besoins fonctionnels

2.3 Besoins non fonctionnels

Exigence	Critère mesurable
Simplicité	installation par une seule commande ; aucune dépendance externe (pas de base de données à administrer).
Réactivité	cycle de collecte de 5 s ; interface mise à jour toutes les 2 s.
Fiabilité	aucune notification perdue en cas de panne d'un canal ou de redémarrage ; résolution automatique des alertes.
Sécurité	API fermée par défaut ; aucun mot de passe par défaut ; accès Docker minimal ; conteneurs sans privilèges.
Maintenabilité	tests automatisés exécutés à chaque modification ; schéma de base versionné.
Portabilité	tout hôte disposant de Docker (Linux, Windows, macOS ; x86-64 et ARM64).
Sobriété	rétention limitée (24 h brutes, 7 jours agrégées) pour maîtriser la taille de la base.
Ergonomie	statuts lisibles sans dépendre de la couleur ; utilisable au clavier ; affichage adapté aux petits écrans.

Tab. 2.3 : Besoins non fonctionnels

2.4 Diagramme de cas d'utilisation global

La figure 2.1 représente les interactions entre les acteurs et le système.

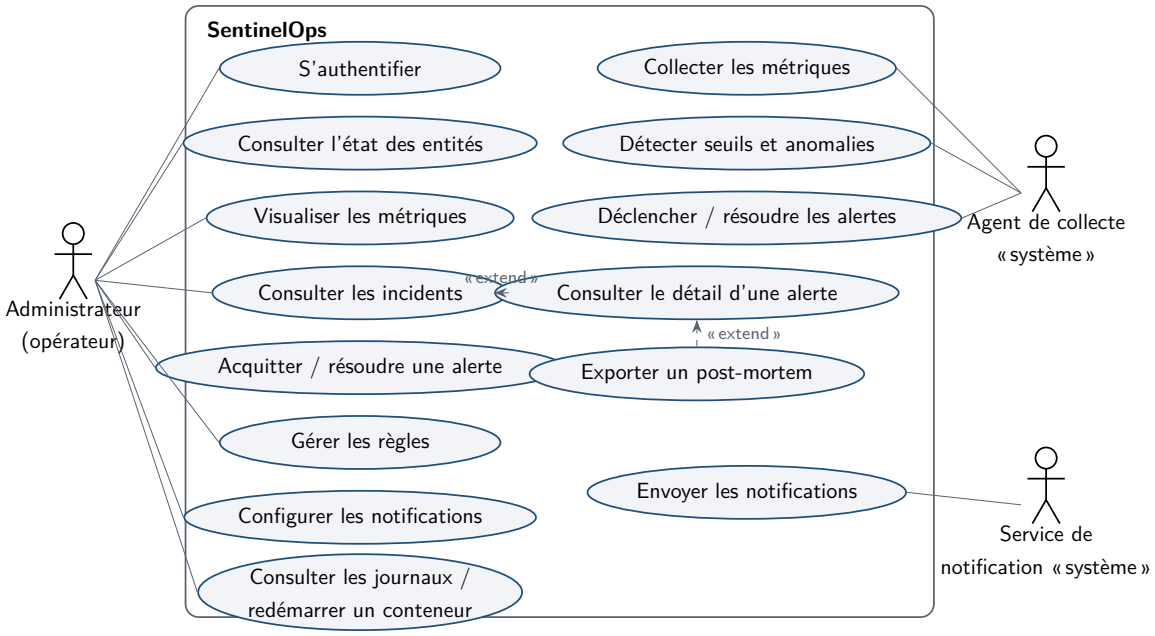


Fig. 2.1 : Diagramme de cas d'utilisation global

2.5 Description des cas d'utilisation principaux

Cas d'utilisation : S'authentifier	
Acteur	Administrateur
Préconditions	la plateforme est démarrée.
Scénario nominal	1. L'administrateur saisit son identifiant et son mot de passe. 2. Le système vérifie l'empreinte du mot de passe. 3. Le système crée une session et ouvre le tableau de bord.
Alternatives	2a. Identifiants incorrects : message d'erreur ; après 5 échecs en 15 minutes depuis la même adresse, la connexion est bloquée temporairement. 3a. Première connexion : le système impose le choix d'un nouveau mot de passe d'au moins 8 caractères avant tout accès.
Postconditions	une session valable 12 heures (2 heures d'inactivité au plus) est ouverte.

Tab. 2.4 : Description du cas « S'authentifier »

Cas d'utilisation : Traiter un incident	
Acteur	Administrateur
Préconditions	l'administrateur est connecté ; une alerte est déclenchée.
Scénario nominal	1. L'incident apparaît en tête de la liste. 2. L'administrateur ouvre le détail de l'alerte. 3. Il consulte la cause (valeurs mesurées, courbe, processus). 4. Il acquitte l'alerte. 5. Une fois le problème corrigé, l'alerte se résout automatiquement et une notification de résolution est envoyée.
Alternatives	4a. Il résout l'alerte manuellement. 4b. Pour un conteneur, il consulte ses journaux ou le redémarre. 5a. Il exporte le post-mortem au format Markdown.
Postconditions	l'alerte est résolue et son historique reste consultable 30 jours.

Tab. 2.5 : Description du cas « Traiter un incident »

Cas d'utilisation : Gérer une règle	
Acteur	Administrateur
Préconditions	l'administrateur est connecté.
Scénario nominal	1. Il ouvre l'onglet « Règles ». 2. Il saisit l'identifiant, la métrique, l'opérateur, le seuil, la durée et la sévérité. 3. Le système valide les champs et enregistre la règle. 4. L'agent l'applique dès le cycle suivant.
Alternatives	3a. Champ invalide : le système refuse avec un message précis. 4a. Désactivation ou suppression : les alertes ouvertes de la règle sont résolues au cycle suivant, l'historique est conservé.
Postconditions	la règle est active et visible dans la liste.

Tab. 2.6 : Description du cas « Gérer une règle »

Conclusion

Les besoins étant spécifiés, le chapitre suivant présente la conception retenue pour y répondre : architecture, données, automate d'alertes et détection d'anomalies.

CHAPITRE 3

Conception

Introduction

Ce chapitre présente les choix de conception de SentinelOps. Il décrit l'architecture globale, la conception de l'agent et de son automate d'alertes, la méthode de détection d'anomalies, le modèle de données et les principales interactions sous forme de diagrammes de séquence.

3.1 Architecture globale

SentinelOps s'exécute sur un seul hôte, sous forme de cinq conteneurs Docker. Deux font le travail principal, deux protègent l'accès à Docker et un prépare le volume de données (figure 3.1).

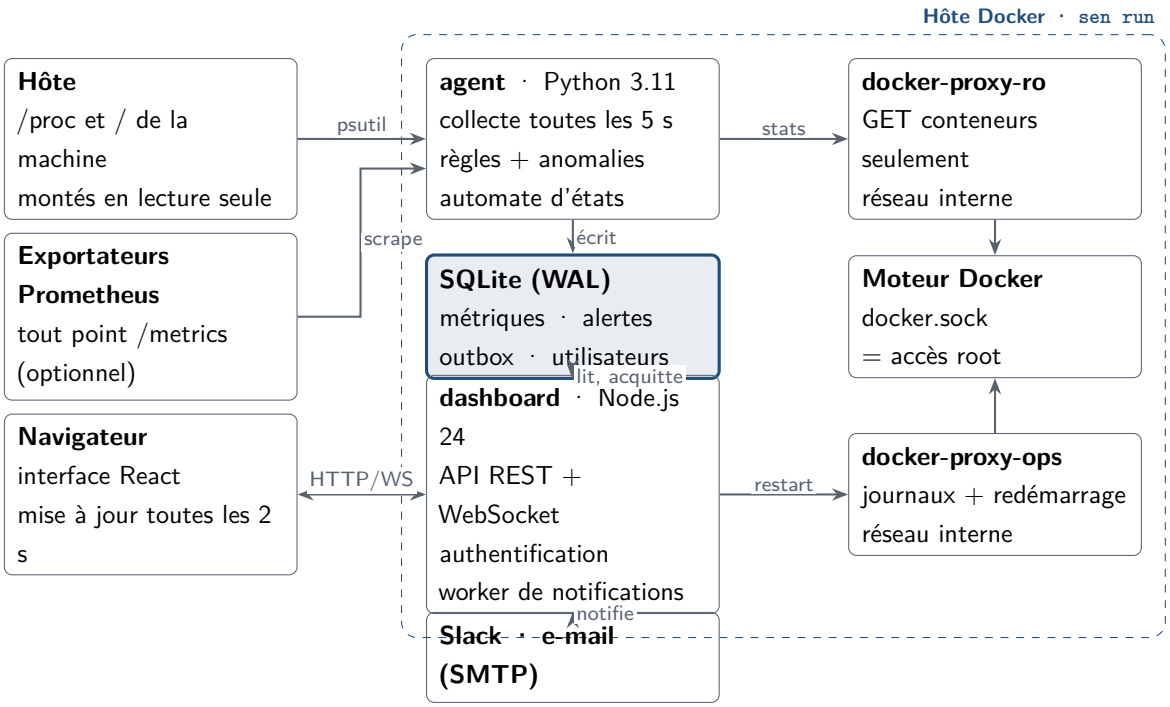


Fig. 3.1 : Architecture globale de SentinelOps

Conteneur	Technologie	Responsabilité
`agent`	Python 3.11 (asyncio)	collecter, détecter, faire évoluer les alertes, nettoyer les anciennes données
`dashboard`	Node.js 24, Express, React	interface web, API REST, authentification, WebSocket, envoi des notifications
`docker-proxy-ro`	proxy HAProxy	n'autorise que la lecture des conteneurs pour l'agent
`docker-proxy-ops`	proxy HAProxy	n'autorise que les journaux et le redémarrage pour le dashboard
`volume-init`	shell (exécution unique)	attribue le volume à l'utilisateur non privilégié (uid 1000)

Tab. 3.1 : Rôle des conteneurs

Principe d'architecture

L'agent et le dashboard ne s'appellent jamais directement : la **base SQLite est le contrat** entre eux. L'agent écrit les métriques et les alertes, le dashboard les lit et enregistre les actions des opérateurs. Chacun peut redémarrer sans interrompre l'autre.

Les règles de conception suivantes ont guidé la refonte de la version 0.2 :

Règle	Problème résolu
Un seul schéma, défini par des migrations SQL versionnées partagées	le schéma défini à trois endroits en v0.1 : une colonne utilisée n'existait nulle part
Un seul écrivain par groupe de colonnes	concurrence et responsabilités floues
Outbox transactionnelle pour les notifications	notifications perdues ou envoyées pour une alerte annulée
Automate d'états avec durée et index unique	alertes bruyantes et dupliquées
Interface `Detector` commune	détecteur d'anomalies inutilisé ; extensibilité vers l'apprentissage automatique
API fermée par défaut	API entièrement ouverte en v0.1
Accès Docker par proxies au moindre privilège	socket Docker monté en lecture-écriture
Horodatages en millisecondes UTC	formats de date mélangés

Tab. 3.2 : Règles de conception de la version 0.2

3.2 Conception de l'agent

3.2.1 Chaîne de traitement

L'agent exécute une boucle à cadence fixe de 5 secondes. Les opérations lentes (appels à Docker, lecture des processus) s'exécutent dans des fils séparés afin de ne jamais bloquer la boucle (figure 3.2).

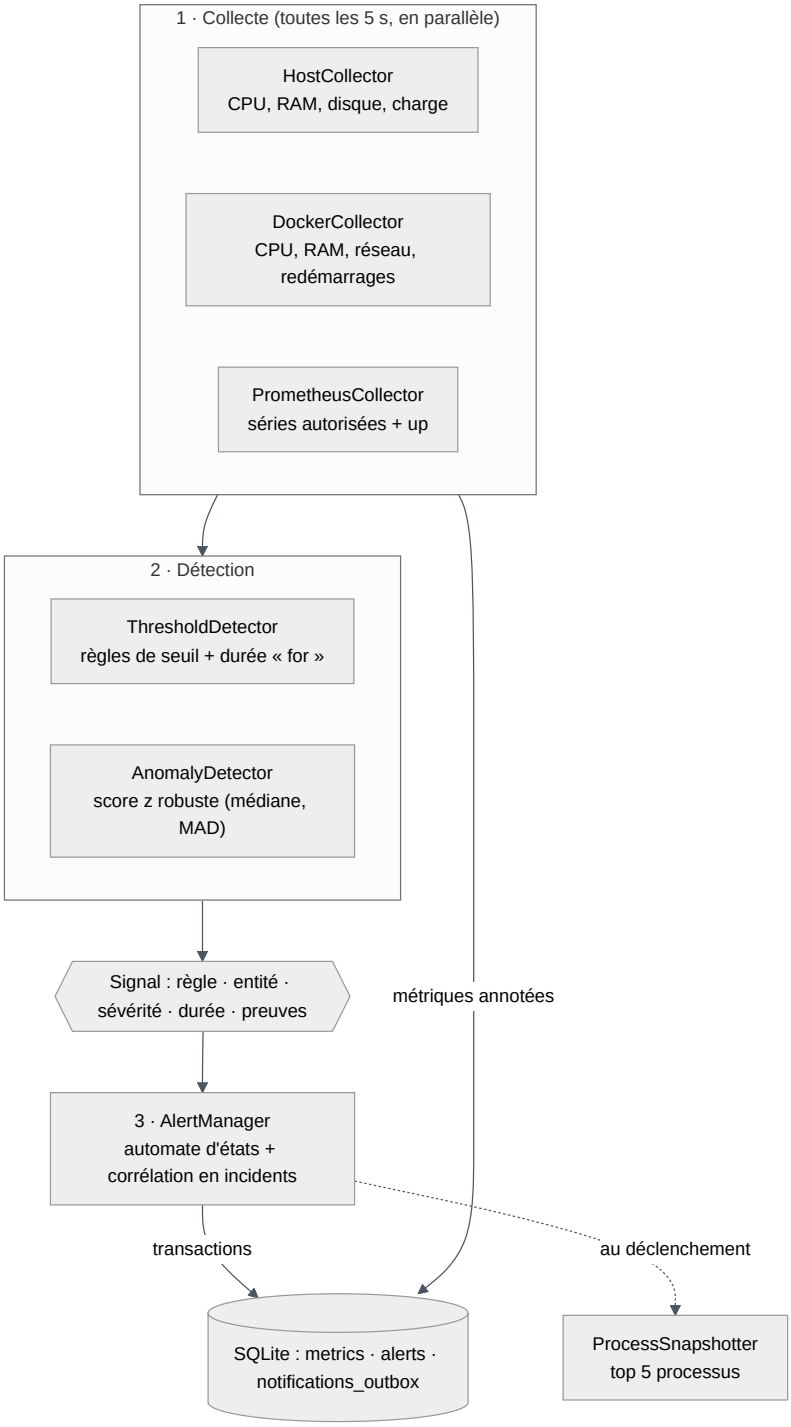


Fig. 3.2 : Chaîne de traitement de l'agent

3.2.2 Diagramme de classes

Le contrat central est la classe `Signal` : un détecteur, quel qu'il soit, ne produit que des signaux, et le gestionnaire d'alertes ne consomme que des signaux. Ajouter un nouveau détecteur (par exemple un modèle Isolation Forest prévu en v0.2) ne demande donc aucune modification du cycle de vie des alertes (figure 3.3).

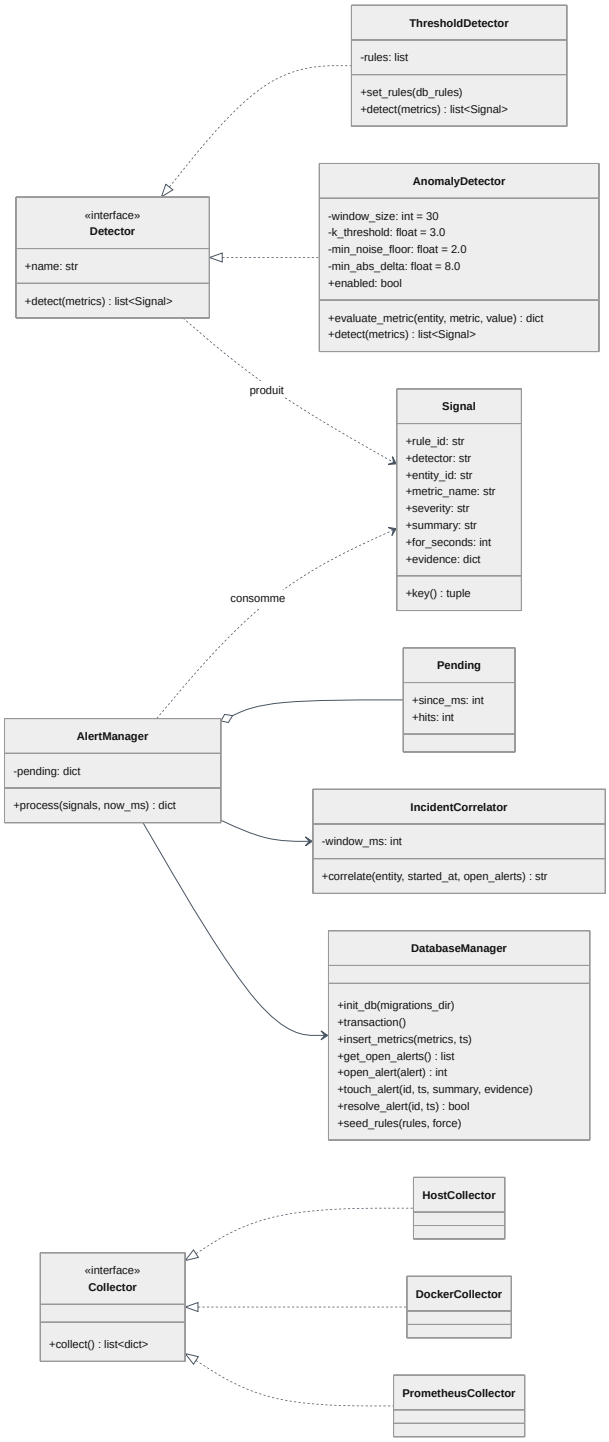


Fig. 3.3 : Diagramme de classes de l'agent

3.2.3 Automate d'états d'une alerte

Le cycle de vie suit le modèle des règles d'alerte de Prometheus [2] (figure 3.4).

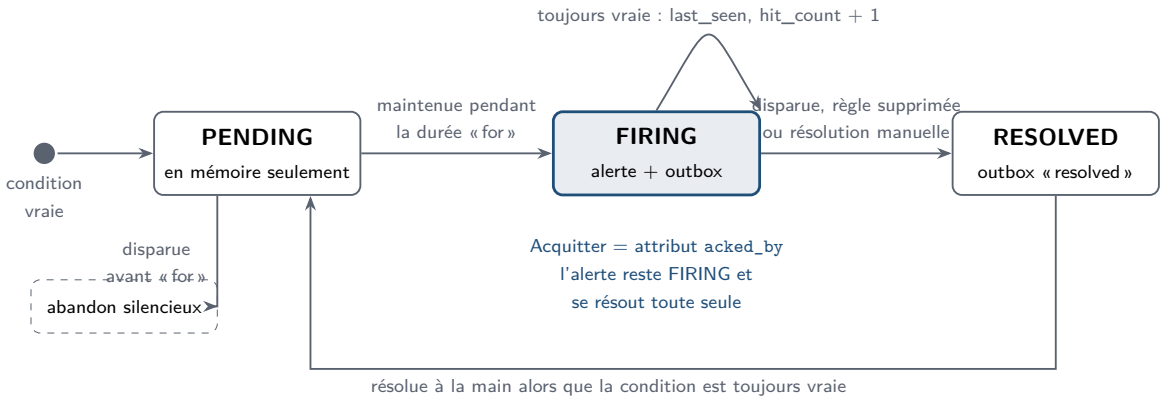


Fig. 3.4 : Automate d'états d'une alerte

- **PENDING** : la condition vient d'apparaître. L'état est conservé *uniquement en mémoire* : rien n'est écrit en base et personne n'est notifié. Si la condition disparaît avant la durée `for`, elle est abandonnée silencieusement : c'est ce qui supprime le bruit des pics passagers.
- **FIRING** : la condition a duré au moins `for` secondes. Une transaction unique écrit l'alerte et une ligne de notification. Les cycles suivants mettent seulement à jour `last_seen`, le compteur et les dernières preuves.
- **RESOLVED** : la condition a disparu (ou la règle a été supprimée). Une transaction marque l'alerte résolue (`resolved_by = auto`) et crée une notification de résolution.
- L'**acquittement** n'est pas un état mais un attribut (`acked_by`, `acked_at`) : une alerte acquittée reste FIRING et se résout automatiquement.

Un **index unique partiel** sur l'empreinte (`sha256(règle:entité)`) des alertes à l'état FIRING garantit au niveau de la base qu'une même règle ne peut pas avoir deux alertes ouvertes sur une même entité, même en cas de concurrence.

3.2.4 Corrélation en incidents

Une nouvelle alerte rejoint l'incident d'une autre alerte ouverte si elles concernent la **même entité** et ont commencé à moins de **120 secondes** d'écart ; sinon un nouvel incident est créé avec un identifiant de la forme `inc-` suivi de 8 caractères hexadécimaux. Par exemple, lors des tests, une alerte de seuil et une alerte d'anomalie sur le CPU du même conteneur, apparues à la même seconde, ont été regroupées dans un seul incident.

3.3 Détection d'anomalies

3.3.1 Principe : le score z robuste

Le détecteur signale une valeur éloignée du comportement récent d'une métrique sur une entité, sans seuil défini à la main. Pour chaque couple (entité, métrique), il conserve les 30 dernières valeurs (2 min 30 s) et, à partir de 10 valeurs, évalue chaque nouvelle mesure x :

$$\tilde{x} = \text{médiane(fenêtre)}, \qquad \sigma = \max(1{,}4826 \times \text{médiane}\left(|x_i - \tilde{x}|\right),\ 2), \qquad z = \frac{x - \tilde{x}}{\sigma} \tag{3.1}$$

La valeur est anormale si $|z| > 3$ **et** si elle s'écarte d'au moins 8 unités de la médiane. Le facteur 1,4826 rend l'écart absolu médian (MAD) comparable à un écart-type pour des données normales [8, 9]. Les deux garde-fous (plancher de σ et écart minimal) évitent les fausses alertes sur des métriques presque plates. La plage attendue $\tilde{x} \pm 3\sigma$ est enregistrée avec chaque mesure et affichée sur les courbes.

3.3.2 Exemple de calcul

Dix dernières valeurs du CPU d'un conteneur : 20, 21, 19, 20, 22, 20, 21, 19, 20, 21 ; nouvelle valeur : 75.

1. Médiane $\tilde{x} = 20$.
2. Écarts à la médiane : 0, 1, 1, 0, 2, 0, 1, 1, 0, 1 ; leur médiane vaut 1, donc $1{,}4826 \times 1 = 1{,}48$, relevé au plancher $\sigma = 2$.
3. $z = (75 - 20)/2 = 27{,}5 > 3$ et l'écart de 55 dépasse 8 : la valeur est anormale.
4. Une alerte n'est créée que si l'anomalie dure 15 secondes (3 cycles), exactement comme la durée `for` d'une règle.

3.3.3 Pourquoi la médiane plutôt que la moyenne

Avec la moyenne et l'écart-type, une anomalie durable « se cache » : le saut lui-même augmente l'écart-type, ce qui fait chuter le score dès les mesures suivantes. La médiane et le MAD ne sont presque pas influencés par quelques valeurs extrêmes. Ce choix a été validé par une mesure présentée au chapitre 5 (section 5.6).

3.4 Modèle de données

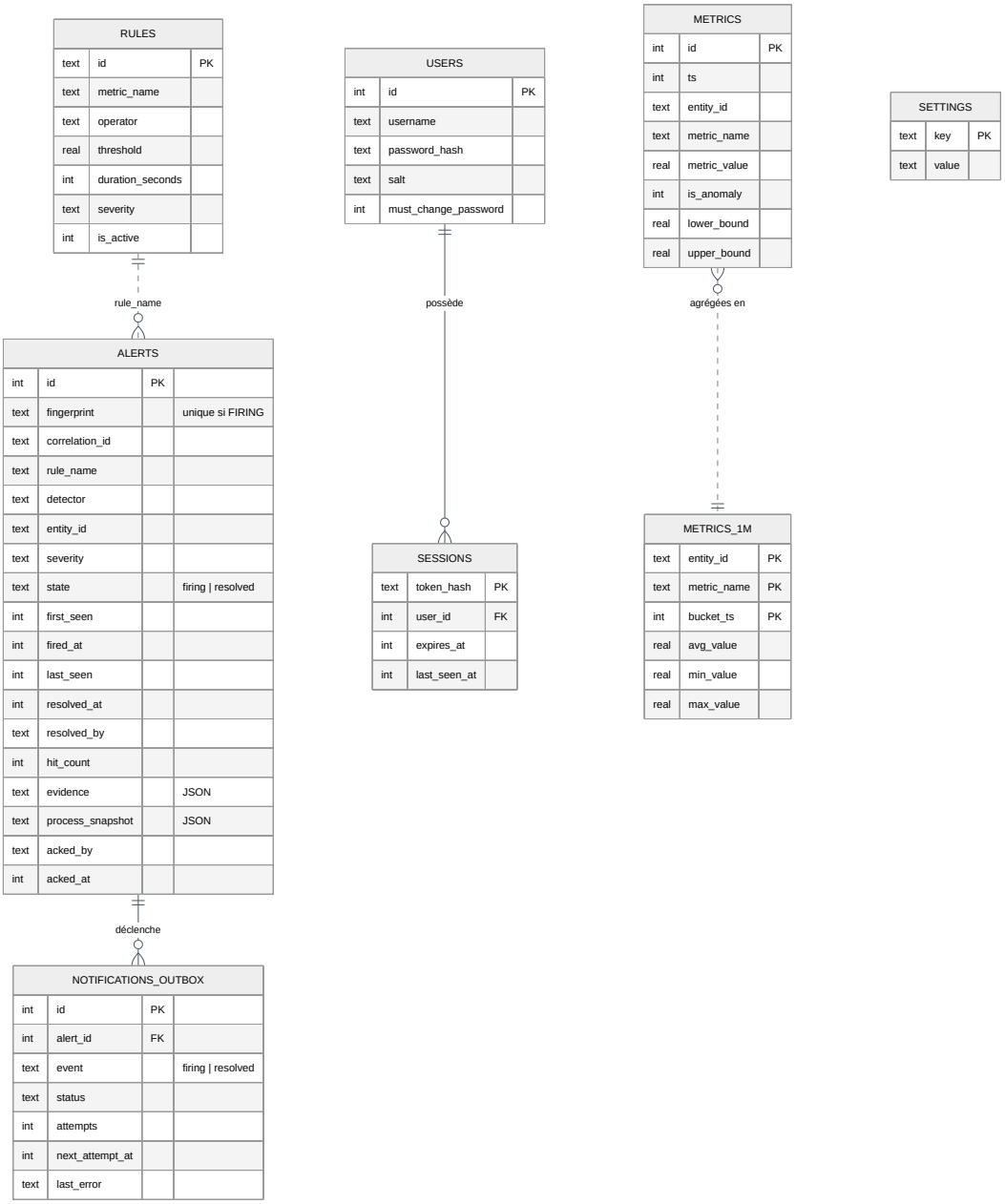


Fig. 3.5 : Modèle de données (principales tables)

Toutes les dates sont des entiers en millisecondes UTC. La base fonctionne en mode WAL (*write-ahead logging*), qui permet au dashboard de lire pendant que l'agent écrit [10].

Table	Écrite par	Conservation
`metrics`	agent	24 heures
`metrics_1m`	agent (agrégats par minute)	7 jours
`alerts`	agent (détection) et dashboard (acquittement, résolution manuelle)	30 jours après résolution
`notifications_outbox`	agent et dashboard, dans la même transaction que l'alerte	7 jours après traitement
`rules`, `settings`	dashboard (initialisation des règles par l'agent)	permanente
`users`, `sessions`	dashboard	sessions expirées supprimées
`schema_version`, `app_meta`	les deux / agent	permanente

Tab. 3.3 : Propriété et conservation des tables

3.4.1 Migrations versionnées

Le schéma est défini une seule fois, dans des fichiers SQL numérotés (`shared/migrations/001_init.sql`). L'agent (Python) et le dashboard (Node.js) exécutent les mêmes fichiers au démarrage, sous un verrou d'écriture exclusif (`BEGIN IMMEDIATE`) : celui qui démarre en premier applique les migrations manquantes, l'autre constate qu'il n'y a rien à faire. La migration 001 met également à niveau une base de la version 0.1 en conservant les utilisateurs, les règles et les paramètres.

3.4.2 Outbox transactionnelle

Enregistrer une alerte et envoyer sa notification sont deux actions distinctes : un arrêt entre les deux ferait perdre la notification. Le patron *transactional outbox* [11] résout ce problème : l'alerte et une ligne de notification sont écrites dans la même transaction, puis un processus indépendant livre les notifications en attente et réessaie en cas d'échec. La livraison est dite « au moins une fois » : un doublon est possible après un échec partiel, une perte ne l'est pas.

3.5 Diagrammes de séquence

3.5.1 Cycle de collecte et de détection

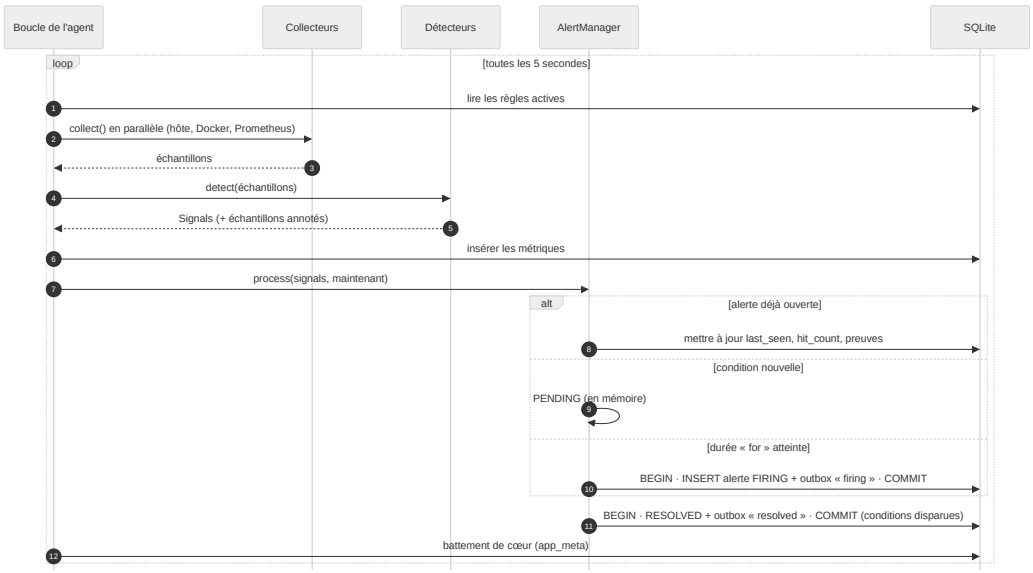


Fig. 3.6 : Diagramme de séquence du cycle de détection

3.5.2 Livraison des notifications

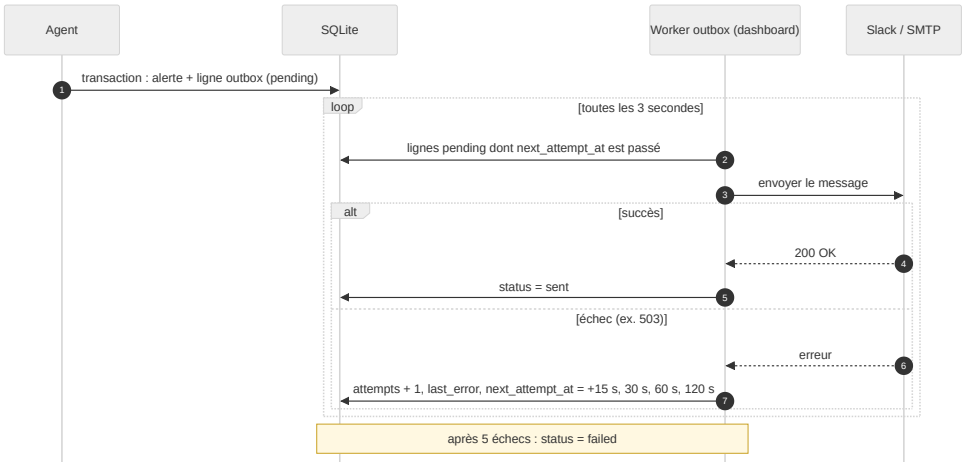


Fig. 3.7 : Diagramme de séquence de la livraison des notifications

3.5.3 Authentification et flux temps réel

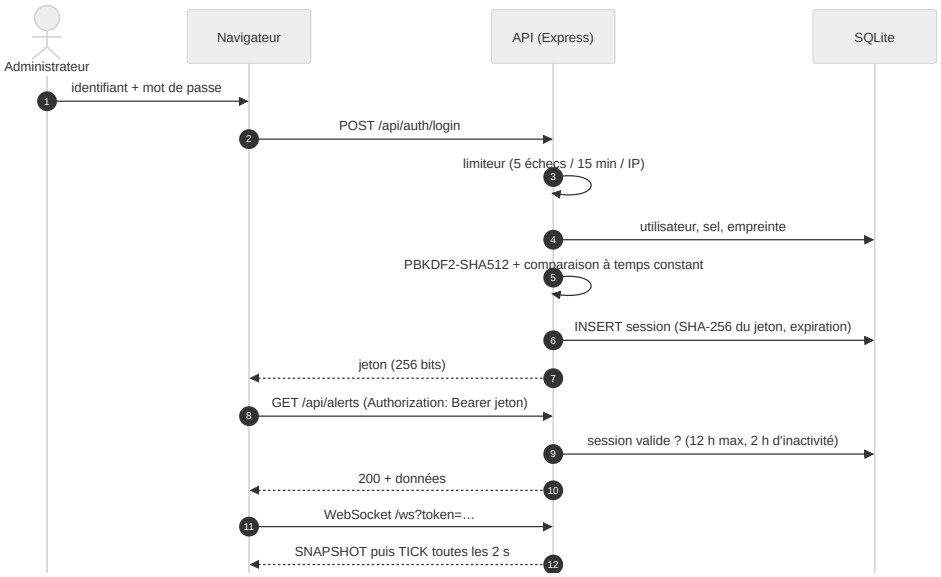


Fig. 3.8 : Diagramme de séquence de l'authentification

3.5.4 Acquittement et résolution manuelle

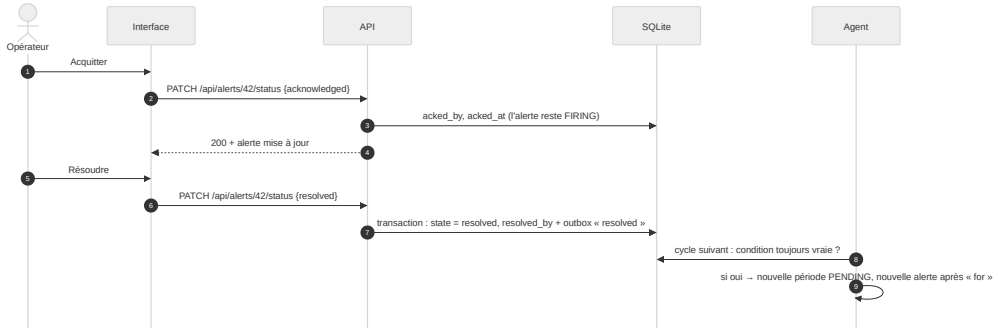


Fig. 3.9 : Diagramme de séquence de l'acquittement et de la résolution

Conclusion

La conception repose sur un contrat simple entre deux programmes (la base de données), un cycle de vie des alertes explicite et une détection extensible. Le chapitre suivant présente sa réalisation.

CHAPITRE 4

Réalisation

Introduction

Ce chapitre présente la mise en œuvre de SentinelOps : l'environnement de travail, les choix technologiques, les extraits de code les plus importants, le lanceur en une commande et les interfaces de l'application.

4.1 Environnement de travail

4.1.1 Environnement matériel

Élément	Caractéristique
Poste de développement	**[marque et modèle du PC]**
Processeur	**[ex. Intel Core i5, 8 cœurs]**
Mémoire vive	**[ex. 16 Go]**
Système d'exploitation	Windows avec Docker Desktop (WSL 2)
Serveur de test	instance cloud Linux (Ubuntu), **[fournisseur et taille de l'instance]**

Tab. 4.1 : Environnement matériel

4.1.2 Environnement logiciel

Outil	Version	Utilisation
Python	3.11	agent de supervision (asyncio, psutil, docker, PyYAML)
Node.js	24	serveur du dashboard (Express, ws, better-sqlite3)
React, TypeScript	18, 5	interface utilisateur (Vite, Recharts)
SQLite	3 (mode WAL)	stockage partagé
Docker, Docker Compose	27+, v2	conteneurisation et orchestration locale
docker-socket-proxy	0.5.0	accès restreint à l'API Docker
pytest, node :test	—	tests automatisés
GitHub Actions	—	intégration continue
Trivy	—	analyse de vulnérabilités des images
Visual Studio Code, Git	—	édition du code et gestion de versions

Tab. 4.2 : Environnement logiciel

4.2 Choix technologiques

Besoin	Choix	Justification
Agent de collecte	Python + asyncio	bibliothèques mûres pour le système (`psutil`) et Docker ; une boucle asynchrone suffit pour une cadence de 5 s
Serveur web	Node.js + Express	un seul langage avec l'interface ; WebSocket natif via `ws`
Interface	React + TypeScript	composants réutilisables, typage des données de l'API
Stockage	SQLite (WAL)	aucun serveur à installer, un fichier sur un volume, lectures concurrentes
Déploiement	Docker Compose	une seule commande, identique sur un PC et sur un serveur
Graphiques	Recharts	courbes temps réel avec bande attendue

Tab. 4.3 : Choix technologiques

Pourquoi pas PostgreSQL ou Kafka ?

SentinelOps vise un seul hôte et une installation en une commande. Un serveur de base de données ou un bus de messages aurait ajouté des conteneurs à configurer, ce que le projet cherche justement à éviter. Le passage à PostgreSQL reste possible grâce aux migrations SQL standard (voir les perspectives).

4.3 Organisation du code

```
sentinelops/
|-- agent/ collecteurs, detecteurs, gestion des alertes (Python)
| |-- collectors/ hote, Docker, processus, Prometheus
| |-- engine/ regles, anomalies, alert_manager, correlateur, retention
| |-- db/ migrations et acces SQLite
| `-- tests/ tests pytest
|-- server/ API REST, authentification, WebSocket, outbox (Node.js)
|-- ui/ interface React + TypeScript
|-- shared/migrations schema SQL versionne, partage par les deux programmes
|-- scripts/sen.py lanceur "sen run"
|-- docker-compose.yml
`-- .github/workflows integration continue
```

Extrait 4.1: Arborescence simplifiée du dépôt

4.4 Extraits de code significatifs

4.4.1 Cycle de vie des alertes

L'extrait 4.2 montre le cœur de l'automate : une condition reste en attente tant que sa durée `for` n'est pas atteinte, puis l'alerte est ouverte. L'index unique protège contre une ouverture concurrente.

```
p = self.pending.get(key)
if p is None:
    p = self.pending[key] = Pending(since_ms=now_ms, hits=0)
p.hits += 1

if now_ms - p.since_ms < sig.for_seconds * 1000:
    stats["pending"] += 1
    continue # not long enough yet: nothing written

# PENDING -> FIRING (alert + outbox row in one transaction)
try:
    alert["id"] = await self.db.open_alert(alert)
except sqlite3.IntegrityError:
    continue # unique index: already open elsewhere
del self.pending[key]
```

Extrait 4.2: Passage de PENDING à FIRING (alert_manager.py)

4.4.2 Score z robuste

```
mean = statistics.median(samples) # robust centre
stdev = 1.4826 * statistics.median(abs(x - mean) for x in samples)
effective = max(stdev, self.min_noise_floor) # floor = 2
z = (value - mean) / effective
is_outlier = (abs(z) > self.k_threshold # k = 3
            and abs(value - mean) >= self.min_abs_delta) # >= 8 units
```

Extrait 4.3: Calcul du score z robuste (anomaly_detector.py)

4.4.3 Migrations sous verrou exclusif

```python
conn.execute("PRAGMA journal_mode=WAL")
conn.execute("BEGIN IMMEDIATE") # take the write lock first
current = conn.execute(
    "SELECT COALESCE(MAX(version), 0) FROM schema_version").fetchone()[0]
for version, name, path in list_migrations(migrations_dir):
    if version <= current:
        continue
    for stmt in _split_statements(open(path).read()):
        conn.execute(stmt)
    conn.execute("INSERT INTO schema_version VALUES (?, ?, ?)",
                 (version, name, now_ms()))
conn.execute("COMMIT")
```

Extrait 4.4: Application des migrations (migrate.py)

4.4.4 API fermée par défaut

```javascript
const PUBLIC = new Set(['POST /auth/login', 'GET /health']);

export function requireAuth(req, res, next) {
  const key = `${req.method} ${req.path}`;
  if (PUBLIC.has(key)) return next();

  const user = authService.validateToken(bearerToken(req));
  if (!user) return res.status(401).json({ error: 'Authentication required' });

  if (user.must_change_password && !ONBOARDING.has(key))
    return res.status(403).json({ code: 'PASSWORD_CHANGE_REQUIRED' });
  req.user = user;
  next();
}
```

Extrait 4.5: Intergiciel d'authentification (middleware/auth.js)

Une nouvelle route est donc protégée sans qu'on ait à y penser : il faudrait l'ajouter explicitement à la liste `PUBLIC` pour la rendre accessible sans session.

4.4.5 Livraison des notifications

```javascript
try {
  const status = await channel.dispatch(row, row.event);
  markProcessed(row.outbox_id, status); // 'sent' or 'skipped'
```

```
} catch (err) {
  const attempts = row.attempts + 1;
  const giveUp = attempts >= MAX_ATTEMPTS; // 5
  db.prepare(`UPDATE notifications_outbox
              SET status = ?, attempts = ?, last_error = ?, next_attempt_at = ?
              WHERE id = ?`)
    .run(giveUp ? 'failed' : 'pending', attempts, err.message,
        Date.now() + 15000 * 2 ** (attempts - 1), row.outbox_id); // 15/30/60/120
            s
}
```

Extrait 4.6: Nouvel essai avec délai croissant (services/outbox.js)

4.5 Lanceur en une commande

Le lanceur `sen` (script Python sans dépendance, avec des raccourcis `sen` et `sen.cmd`) masque Docker Compose. La commande `sen run` vérifie Docker, génère un mot de passe administrateur au premier lancement, construit et démarre les conteneurs, puis attend que `/api/health` réponde **et** que l'agent ait envoyé son premier signal de vie avant d'afficher « Ready » avec l'adresse et les identifiants (figure 4.1).

Commande	Effet
`sen run`	construit, démarre et attend que tout soit prêt
`sen run --public --port 8080`	expose le dashboard sur le réseau, sur un autre port
`sen status`	état des conteneurs, de l'API et de l'agent
`sen logs [agent|dashboard]`	journaux en continu
`sen password`	affiche le mot de passe administrateur initial
`sen update`, `sen restart`, `sen stop`	mise à jour, redémarrage, arrêt

Tab. 4.4 : Commandes du lanceur `sen`

Capture à insérer

`figures/captures/cap-18-sen-run.png`

Sortie de `sen run` jusqu'au message « Ready »

Fig. 4.1 : Sortie de `sen run` jusqu'au message « Ready »

Capture à insérer

`figures/captures/cap-19-sen-status.png`

Sortie de `sen status`

Fig. 4.2 : Sortie de `sen status`

4.6 Interfaces de l'application

4.6.1 Authentification

Au premier lancement, l'administrateur se connecte avec le mot de passe généré puis doit en choisir un nouveau : tant que ce n'est pas fait, l'API refuse toutes les autres actions.

cap-02-login.png

(a) Page de connexion

cap-03-premier-mot-de-passe.png

(b) Changement obligatoire du mot de passe

Fig. 4.3 : Authentification

4.6.2 Vue d'ensemble

La vue d'ensemble affiche l'état de l'hôte et de chaque conteneur, les incidents ouverts et les courbes temps réel reçues par WebSocket.

Capture à insérer

`figures/captures/cap-04-vue-ensemble-normal.png`

Vue d'ensemble : système en état normal

Fig. 4.4 : Vue d'ensemble : système en état normal

Capture à insérer

`figures/captures/cap-05-vue-ensemble-incident.png`

Vue d'ensemble pendant un incident

Fig. 4.5 : Vue d'ensemble pendant un incident

4.6.3 Détail d'un incident

Un clic sur un incident ouvre un panneau qui répond aux questions de l'opérateur : pourquoi l'alerte s'est déclenchée, ce qui s'est passé, quels processus consommaient les ressources, et un brouillon de compte rendu (*post-mortem*).

Capture à insérer

`figures/captures/cap-06-detail-pourquoi.png`

Détail d'un incident : onglet « Pourquoi » (preuves et valeurs observées)

Fig. 4.6 : Détail d'un incident : onglet « Pourquoi » (preuves et valeurs observées)

Capture à insérer

`figures/captures/cap-07-detail-chronologie.png`

Détail d'un incident : chronologie

Fig. 4.7 : Détail d'un incident : chronologie

Capture à insérer

`figures/captures/cap-08-detail-processus.png`

Détail d'un incident : instantané des processus

Fig. 4.8 : Détail d'un incident : instantané des processus

Capture à insérer

figures/captures/cap-09-detail-postmortem.png

Détail d'un incident : brouillon de post-mortem

Fig. 4.9 : Détail d'un incident : brouillon de post-mortem

4.6.4 Courbe d'anomalie

Capture à insérer

figures/captures/cap-10-courbe-anomalie.png

Courbe d'une métrique avec sa bande attendue et le point anormal

Fig. 4.10 : Courbe d'une métrique avec sa bande attendue et le point anormal

4.6.5 Règles, paramètres et journaux

Capture à insérer

figures/captures/cap-11-regles.png

Gestion des règles d'alerte

Fig. 4.11 : Gestion des règles d'alerte

Capture à insérer

figures/captures/cap-12-parametres.png

Paramètres : canaux de notification et heures calmes

Fig. 4.12 : Paramètres : canaux de notification et heures calmes

Capture à insérer

figures/captures/cap-13-journaux-conteneur.png

Journaux d'un conteneur et bouton de redémarrage

Fig. 4.13 : Journaux d'un conteneur et bouton de redémarrage

4.6.6 Thème clair et affichage mobile

cap-14-theme-clair.png

(a) Thème clair

cap-15-mobile.png

(b) Mobile

Fig. 4.14 : Thème clair et affichage adaptatif

4.6.7 Notifications Slack



(a) Déclenchement



(b) Résolution

Fig. 4.15 : Messages reçus sur Slack

Conclusion

Les fonctionnalités prévues au chapitre 2 sont réalisées et accessibles par une seule commande. Le chapitre suivant décrit comment elles sont sécurisées, testées et déployées.

CHAPITRE 5

Sécurité, tests, intégration continue et déploiement

Introduction

Un outil de supervision a accès à tout l'hôte : il doit lui-même être sûr. Ce chapitre présente les mesures de sécurité, la chaîne d'intégration continue, les tests, le déploiement sur un serveur et l'évaluation de la solution.

5.1 Sécurité de l'application

Risque	Mesure	Vérification
Accès non autorisé à l'API	API fermée par défaut, jeton porteur, seules `POST /api/auth/login` et `GET /api/health` sont publiques	test automatique sur toutes les routes, capture 5.2a
Vol de mots de passe	PBKDF2-SHA512, 100 000 itérations, sel aléatoire [12]	test de connexion
Vol de jeton en base	seule l'empreinte SHA-256 du jeton est stockée	revue de code
Force brute	5 échecs par 15 minutes et par couple (IP, utilisateur)	test automatique
Session oubliée	durée maximale 12 h, inactivité 2 h, révocation à la déconnexion	tests automatiques
Mot de passe initial	généré au premier lancement, changement obligatoire	test automatique
Secrets exposés	secrets en écriture seule : l'API ne renvoie jamais le webhook ni le mot de passe SMTP	test automatique
XSS, clickjacking	en-têtes CSP, `X-Frame-Options: DENY`, `nosniff` [13]	test automatique
Injection via identifiant	identifiants de conteneur validés par expression régulière	test automatique
WebSocket ouvert	jeton exigé à la connexion	test automatique

Tab. 5.1 : Mesures de sécurité de l'application

5.2 Sécurité des conteneurs

Le socket Docker donne un contrôle total de l'hôte. En version 0.1 il était monté directement dans les conteneurs ; en version 0.2, aucun conteneur applicatif n'y a accès. Chacun passe par un proxy [14] qui n'autorise que les appels nécessaires (tableau 5.2), sur un réseau interne sans port publié.

Proxy	Client	Appels autorisés
`docker-proxy-ro`	agent	lecture des conteneurs (liste, inspection, statistiques)
`docker-proxy-ops`	dashboard	journaux et redémarrage ; ni création, ni `exec`, ni suppression

Tab. 5.2 : Proxies d'accès à Docker

Les conteneurs applicatifs suivent les recommandations du NIST SP 800-190 [15] et de Docker [16] :

- exécution sous un utilisateur non privilégié (uid 1000) ;
- système de fichiers en lecture seule, seul `/tmp` est inscriptible en mémoire ;
- suppression de toutes les capacités Linux (`cap_drop: ALL`) et `no-new-privileges` ;
- images multi-étapes : les outils de compilation ne sont pas dans l'image finale ;
- dashboard publié uniquement sur `127.0.0.1` par défaut.

Capture à insérer

`figures/captures/cap-26-docker-ps.png`

Les conteneurs de SentinelOps en fonctionnement (`docker ps`)

Fig. 5.1 : Les conteneurs de SentinelOps en fonctionnement (`docker ps`)

cap-30-curl-401.png

(a) Appel sans jeton : 401

cap-31-proxy-403.png

(b) Appel interdit par le proxy : 403

Fig. 5.2 : Vérification des contrôles d'accès

5.3 Intégration continue

Chaque envoi sur GitHub déclenche le pipeline de la figure 5.3, exécuté par GitHub Actions.

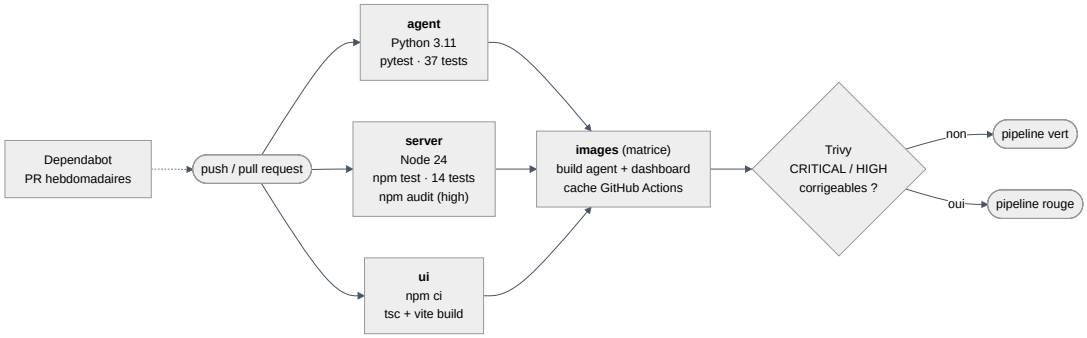


Fig. 5.3 : Pipeline d'intégration continue

- **agent** : installation des dépendances et exécution des tests pytest ;
- **server** : `npm ci`, tests et `npm audit` (échec si une vulnérabilité haute est trouvée) ;
- **ui** : vérification des types TypeScript et construction ;
- **images** : construction des deux images puis analyse par Trivy [17] ; le pipeline échoue en cas de vulnérabilité critique ou haute corrigeable.

Deux mesures protègent la chaîne elle-même : les actions tierces sont épinglées par leur empreinte SHA complète plutôt que par une étiquette modifiable [18], et Dependabot [19] propose chaque semaine la mise à jour des dépendances.

Capture à insérer

`figures/captures/cap-20-ci-vert.png`

Pipeline GitHub Actions réussi

Fig. 5.4 : Pipeline GitHub Actions réussi

Le pipeline a joué son rôle pendant le stage : Trivy a bloqué une image contenant des vulnérabilités dans des paquets du système de base (figure 5.5) et `npm audit` a

signalé une dépendance vulnérable (figure 5.6). Les deux ont été corrigés par mise à jour avant la fusion.

Capture à insérer

`figures/captures/cap-21-trivy-rouge.png`

Analyse Trivy bloquant une image vulnérable

Fig. 5.5 : Analyse Trivy bloquant une image vulnérable

Capture à insérer

`figures/captures/cap-22-npm-audit.png`

Rapport `npm audit`

Fig. 5.6 : Rapport `npm audit`

Capture à insérer

`figures/captures/cap-23-dependabot.png`

Demandes de mise à jour ouvertes par Dependabot

Fig. 5.7 : Demandes de mise à jour ouvertes par Dependabot

5.4 Tests

5.4.1 Tests automatisés

Le projet compte **51 tests automatisés** : 37 pour l'agent (pytest) et 14 pour le serveur (`node:test`). Ils s'exécutent en quelques secondes, sur une base SQLite temporaire, sans Docker.

Fichier	Tests	Ce qui est vérifié
`test_alert_manager.py`	11	attente avant déclenchement, pic court ignoré, pas de doublon, résolution automatique, alerte acquittée, règle supprimée, corrélation, échec de l'instantané sans blocage
`test_detectors.py`	18	opérateurs des seuils, validation de `rules.yaml`, anomalie après apprentissage, séries plates, saut durable, formules Docker, format Prometheus
`test_migrations.py`	4	base neuve, réexécution sans effet, mise à niveau d'une base v0.1, index unique
`test_retention_and_seed.py`	4	agrégation et purge, purge des alertes, initialisation unique des règles
`api.test.js`	14	routes fermées, connexion et limitation, sessions, secrets, règles, cycle de vie, outbox, WebSocket, en-têtes
Total	**51**	

Tab. 5.3 : Répartition des tests automatisés



(a) Agent : `pytest`



(b) Serveur : `npm test`

Fig. 5.8 : Exécution des tests

5.4.2 Scénarios de validation de bout en bout

Scénario	Action	Résultat attendu et obtenu
Charge CPU d'un conteneur	`docker run` d'un conteneur qui boucle (stress)	alerte d'anomalie en environ 15 s, alerte de seuil après 120 s, regroupées dans un même incident
Pic court	charge de 10 secondes	aucune alerte (abandon à l'état PENDING)
Fin de charge	arrêt du conteneur de test	résolution automatique et message de résolution
Webhook indisponible	URL Slack invalide	nouvel essai à 15, 30, 60 puis 120 s, puis échec enregistré
Acquittement	bouton « Acquitter »	l'alerte reste ouverte, sans nouvelle notification, puis se résout seule
Accès sans session	`curl` sur `/api/alerts`	réponse 401
Action Docker interdite	création de conteneur via le proxy	réponse 403

Tab. 5.4 : Scénarios de validation manuelle

5.5 Déploiement sur un serveur

SentinelOps a été installé sur une instance cloud Linux vierge, sans outil d'infrastructure-as-code, afin de vérifier qu'une personne peut l'installer seule à partir du guide (annexe A). La figure 5.9 montre le déploiement obtenu.

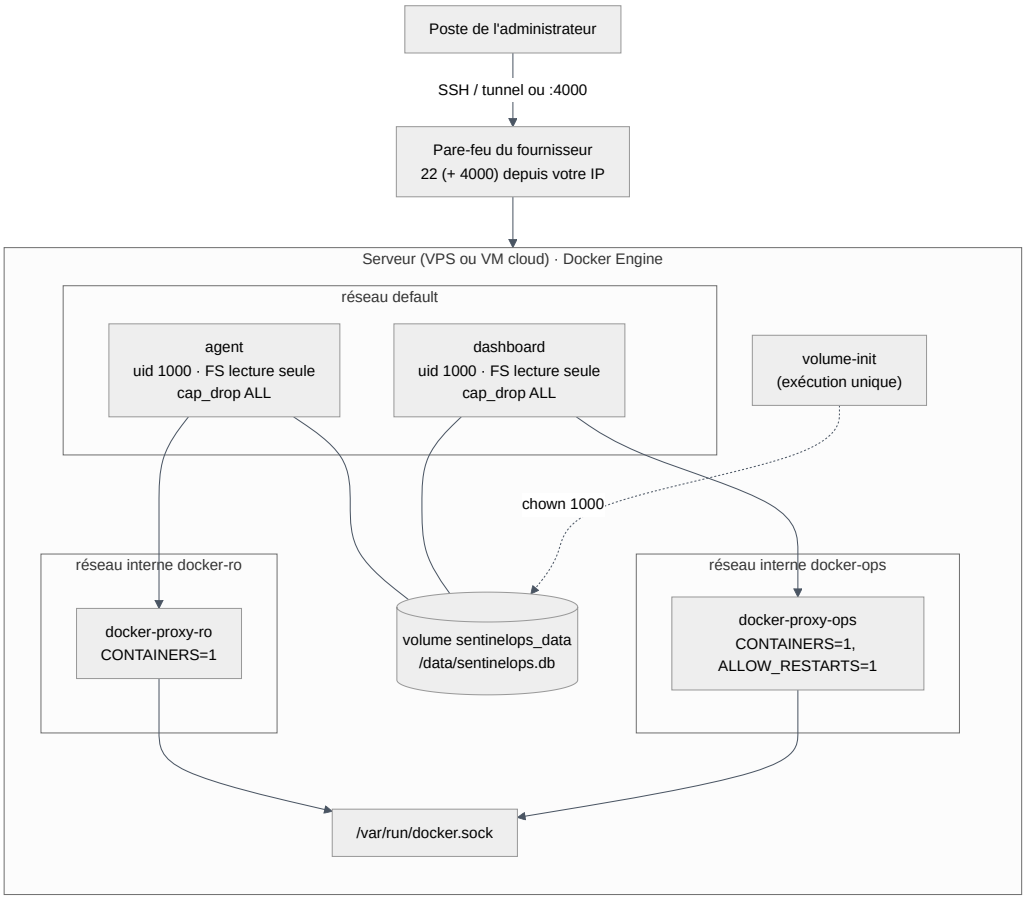


Fig. 5.9 : Diagramme de déploiement

Deux modes d'accès sont possibles :

- **tunnel SSH** (recommandé) : le dashboard reste lié à `127.0.0.1` et aucun port supplémentaire n'est ouvert ; l'opérateur exécute `ssh -L 4000:localhost:4000 utilisateur@serveur` ;
- **port public** : `sen run --public`, avec une règle de pare-feu limitée à l'adresse IP de l'opérateur, de préférence derrière un proxy HTTPS.

Capture à insérer

`figures/captures/cap-27-instance-cloud.png`

Instance cloud utilisée pour le test

Fig. 5.10 : Instance cloud utilisée pour le test

Capture à insérer

`figures/captures/cap-28-pare-feu.png`

Règles du pare-feu (groupe de sécurité) de l'instance

Fig. 5.11 : Règles du pare-feu (groupe de sécurité) de l'instance

Capture à insérer

`figures/captures/cap-29-dashboard-serveur.png`

Dashboard supervisant le serveur distant

Fig. 5.12 : Dashboard supervisant le serveur distant

5.6 Évaluation

5.6.1 Comparaison des deux estimateurs

Pour justifier le choix de la médiane, les deux méthodes ont été appliquées à la même série : 30 mesures autour de 20 % puis un saut durable à 75 %, avec les mêmes paramètres (fenêtre de 30, $k = 3$, plancher 2).

Estimateur	**Mesures signalées**	**Alerte (durée 15 s)**
Moyenne et écart-type	3 (15 s)	non déclenchée
Médiane et MAD	15 (75 s)	déclenchée

Tab. 5.5 : Comportement face à un saut durable

Avec la moyenne, l'anomalie fait elle-même grimper l'écart-type et cesse d'être signalée avant la fin de la durée de 15 secondes : l'incident passe inaperçu. Avec la médiane, elle reste signalée jusqu'à ce que la nouvelle valeur occupe la moitié de la fenêtre, puis devient la nouvelle normale.

5.6.2 Délais de détection et de notification

Ces délais découlent directement de la configuration : cadence de 5 s, durée `for` de la règle, et passage du travailleur d'outbox toutes les 3 s.

Cas	**Passage à FIRING**	**Notification**
Anomalie	15 à 20 s	+ 3 s au plus
CPU hôte (`for` = 60 s)	60 à 65 s	+ 3 s au plus
Mémoire conteneur (`for` = 30 s)	30 à 35 s	+ 3 s au plus

Tab. 5.6 : Délais de détection et de notification

5.6.3 Consommation de ressources

Conteneur	CPU moyen	Mémoire	Taille de l'image
agent	[x %]	[x Mo]	[x Mo]
dashboard	[x %]	[x Mo]	[x Mo]
proxies (×2)	[x %]	[x Mo]	[x Mo]

Tab. 5.7 : Consommation de ressources (relevée avec `docker stats` et `docker images`)

5.6.4 Limites

- un seul hôte est supervisé par installation ;
- SQLite limite le volume de données et l'écriture à un processus à la fois ;
- le dashboard ne gère pas HTTPS lui-même : un proxy inverse est nécessaire en accès public ;
- un seul rôle (administrateur) ; pas de gestion fine des droits ;
- la livraison au moins une fois peut produire un doublon de notification après un échec partiel.

Conclusion

SentinelOps protège l'hôte qu'il supervise, vérifie chaque modification par des tests et une analyse de sécurité automatiques, et s'installe sur un serveur réel en quelques minutes. Ses limites, identifiées clairement, orientent les perspectives présentées en conclusion.

Conclusion générale et perspectives

Ce stage, réalisé au sein de **Barmaj Information Technology**, avait pour objectif de concevoir et réaliser une plateforme de supervision et d'alerte auto-hébergée, simple à installer et fiable. Le point de départ était un constat vécu : mettre en place Prometheus, Alertmanager et Grafana demande de configurer et de synchroniser plusieurs outils, ce qui est long et fragile pour une petite équipe.

SentinelOps répond à ce besoin. Une seule commande, `sen run`, construit et démarre l'ensemble et affiche « Ready » lorsque tout fonctionne. L'outil collecte les métriques de l'hôte, des conteneurs et de cibles Prometheus, détecte les incidents par des règles avec durée et par un score z robuste, regroupe les alertes liées en incidents, explique chaque alerte par des preuves mesurées et un instantané des processus, et notifie les équipes sur Slack ou par courriel sans jamais perdre une notification.

Au-delà des fonctionnalités, le travail a surtout porté sur la qualité. Une revue du prototype initial a relevé seize problèmes, de la base de données définie à trois endroits à une API entièrement ouverte. La version 0.2 les corrige par des choix d'architecture : schéma unique par migrations, automate d'états, outbox transactionnelle, API fermée par défaut, accès à Docker au moindre privilège et conteneurs durcis. Cinquante et un tests automatisés et une chaîne d'intégration continue avec analyse de vulnérabilités vérifient chaque modification.

Sur le plan personnel, ce stage m'a permis de [**compléter : compétences acquises, par exemple la conception d'un système concurrent autour d'une base partagée, les pratiques DevSecOps, la rigueur des tests, le travail en autonomie avec un encadrant**].

Perspectives

Plusieurs évolutions sont prévues dans la feuille de route du projet :

- **détection par apprentissage automatique** : ajouter un détecteur Isolation Forest multivarié, sans modifier le gestionnaire d'alertes grâce à l'interface `Detector` ;
- **indicateurs de fiabilité** : calculer le MTTD et le MTTR à partir des dates déjà enregistrées pour chaque alerte ;

- **supervision de plusieurs hôtes** : des agents distants qui envoient leurs métriques au dashboard ;
- **PostgreSQL** en option pour les volumes plus importants ;
- **déploiement automatisé** avec Terraform et un chart Helm pour Kubernetes ;
- **assistant d'analyse** : un modèle de langage exécuté localement (par exemple via Ollama) pour résumer un incident et proposer des pistes de résolution ;
- **HTTPS intégré et rôles** (lecture seule, opérateur, administrateur).

Bibliographie

[1] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering : How Google Runs Production Systems*. O'Reilly Media, 2016.

[2] Prometheus Authors. Alerting rules. `https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/`. Consulté en 2026.

[3] Prometheus Authors. Alertmanager. `https://prometheus.io/docs/alerting/latest/alertmanager/`. Consulté en 2026.

[4] Grafana Labs. Grafana documentation. `https://grafana.com/docs/grafana/latest/`. Consulté en 2026.

[5] Zabbix SIA. Zabbix documentation. `https://www.zabbix.com/documentation/current/`. Consulté en 2026.

[6] Netdata Inc. Netdata documentation. `https://learn.netdata.cloud/`. Consulté en 2026.

[7] Datadog Inc. Datadog documentation. `https://docs.datadoghq.com/`. Consulté en 2026.

[8] Christophe Leys, Christophe Ley, Olivier Klein, Philippe Bernard, and Laurent Licata. Detecting outliers : Do not use standard deviation around the mean, use absolute deviation around the median. *Journal of Experimental Social Psychology*, 49(4) :764–766, 2013.

[9] Boris Iglewicz and David C. Hoaglin. *How to Detect and Handle Outliers*, volume 16 of *The ASQC Basic References in Quality Control*. ASQC Quality Press, 1993.

[10] SQLite Consortium. Write-ahead logging. `https://www.sqlite.org/wal.html`. Consulté en 2026.

[11] Chris Richardson. Pattern : Transactional outbox. `https://microservices.io/patterns/data/transactional-outbox.html`. Consulté en 2026.

[12] Kathleen Moriarty, Burt Kaliski, and Andreas Rusch. Pkcs #5 : Password-based cryptography specification version 2.1. RFC 8018, Internet Engineering Task Force, 2017.

[13] OWASP Foundation. Owasp top 10. https://owasp.org/www-project-top-ten/. Consulté en 2026.

[14] Tecnativa. docker-socket-proxy. https://github.com/Tecnativa/docker-socket-proxy. Version 0.5.0.

[15] Murugiah Souppaya, John Morello, and Karen Scarfone. Application container security guide. NIST Special Publication 800-190, National Institute of Standards and Technology, 2017.

[16] Docker Inc. Docker engine security. https://docs.docker.com/engine/security/. Consulté en 2026.

[17] Aqua Security. Trivy documentation. https://trivy.dev/. Consulté en 2026.

[18] GitHub. Security hardening for github actions. https://docs.github.com/en/actions/security-for-github-actions/security-guides/security-hardening-for-github-actions. Consulté en 2026.

[19] GitHub. About dependabot version updates. https://docs.github.com/en/code-security/dependabot. Consulté en 2026.

ANNEXE A

Guide d'installation

SentinelOps s'installe de la même façon sur un PC, un VPS ou une instance cloud (AWS EC2, Azure, Google Cloud, OVHcloud, Hetzner, DigitalOcean…). Seuls la création de la machine et le pare-feu dépendent du fournisseur.

Prérequis

- Linux (Ubuntu 22.04 ou 24.04 conseillé), 2 vCPU et 2 Go de RAM au minimum, 10 Go de disque ; sous Windows ou macOS : Docker Desktop ;
- Docker Engine avec le module Compose v2, Git et Python 3.8 ou plus.

Étapes sur un serveur Linux

```
# 1. Docker (script officiel) + droits pour l'utilisateur courant
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER && newgrp docker

# 2. Recuperer le projet
git clone https://github.com/Amin-BenZid/SentinelOps.git
cd SentinelOps

# 3. Demarrer
./sen run # acces par tunnel SSH (recommande)
./sen run --public # ou acces direct sur le port 4000
```

Extrait A.1: Installation sur un serveur Linux

Accès au dashboard

- **Tunnel SSH** : depuis le poste de l'opérateur, `ssh -L 4000:localhost:4000 utilisateur@IP_DU_SERVEUR`, puis ouvrir `http://localhost:4000`.

- **Port public** : autoriser le port TCP 4000 dans le pare-feu du fournisseur (groupe de sécurité AWS, NSG Azure, règle VPC Google Cloud, pare-feu du VPS) **uniquement depuis l'adresse IP de l'opérateur**, puis ouvrir `http://IP_DU_SERVEUR:4000`.

Le mot de passe initial s'affiche à la fin de `sen run` (ou avec `sen password`) ; il doit être changé à la première connexion.

ANNEXE B

Référence de l'API REST

Toutes les routes sont préfixées par `/api` et exigent l'en-tête `Authorization: Bearer <jeton>`, sauf les deux routes publiques.

Tab. B.1 : Routes de l'API REST

Méthode	Route	Rôle
POST	`/auth/login`	connexion (publique, limitée en débit)
GET	`/health`	état de l'API et de l'agent (publique)
GET	`/auth/me`	utilisateur courant
POST	`/auth/change-credentials`	changer l'identifiant ou le mot de passe
POST	`/auth/logout`	révoquer le jeton
GET	`/alerts`	liste des alertes (filtres)
PATCH	`/alerts/:id/status`	acquitter, désacquitter, résoudre, rouvrir
DELETE	`/alerts/purge`	supprimer les alertes résolues
GET	`/metrics/history`	historique d'une métrique sur une période
GET	`/filters`	valeurs disponibles pour les filtres
GET	`/services`	état des entités supervisées
GET, POST	`/rules`	lister, créer ou modifier une règle
PATCH	`/rules/:id/toggle`	activer ou désactiver une règle
DELETE	`/rules/:id`	supprimer une règle
GET, POST	`/settings`	lire ou modifier les paramètres (secrets en écriture seule)
POST	`/test/slack`	envoyer un message de test

Méthode	Route	Rôle
GET	`/notifications`	historique de l'outbox
GET	`/containers/:id/logs`	journaux d'un conteneur
POST	`/containers/:id/restart`	redémarrer un conteneur

Le flux temps réel est disponible en WebSocket sur `/ws?token=<jeton>`.

ANNEXE C

Règles d'alerte par défaut

Le fichier `agent/config/rules.yaml` initialise les règles au premier démarrage ; elles sont ensuite gérées depuis l'interface.

```yaml
version: "0.2"
scan_interval_seconds: 5

rules:
  - id: "host_high_cpu"
    metric: "cpu_percent"
    operator: ">"
    threshold: 85.0
    duration_seconds: 60
    severity: "critical"
    summary: "Host CPU utilization exceeded 85%"

  - id: "docker_container_oom"
    metric: "docker_memory_percent"
    operator: ">"
    threshold: 90.0
    duration_seconds: 30
    severity: "critical"
    summary: "Container memory close to its limit"
```

Extrait C.1: Extrait de rules.yaml

Règle	Métrique	Op.	Seuil	Durée	Gravité
host_high_cpu	cpu_percent	>	85	60 s	critique
host_high_mem	memory_percent	>	80	60 s	avertissement
host_disk_full	disk_percent	>	90	300 s	critique
docker_container_oom	docker_memory_percent	>	90	30 s	critique
docker_container_cpu	docker_cpu_percent	>	90	120 s	avertissement

Tab. C.1 : Règles livrées par défaut

ANNEXE D

Variables d'environnement

Tab. D.1 : Variables d'environnement

Variable	Défaut	Rôle
DASHBOARD_BIND	127.0.0.1	adresse d'écoute publiée (0.0.0.0 avec --public)
DASHBOARD_PORT	4000	port publié du dashboard
ADMIN_USERNAME	admin	identifiant administrateur initial
ADMIN_PASSWORD	généré	mot de passe initial
SESSION_TTL_HOURS	12	durée maximale d'une session
SESSION_IDLE_MINUTES	120	déconnexion après inactivité
SLACK_WEBHOOK_URL	—	webhook Slack (modifiable dans l'interface)
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS	–, 587	serveur de courriel
ALERT_EMAIL_TO	—	destinataire des courriels d'alerte
HOST_ID	local-host	nom de l'hôte dans le dashboard
ANOMALY_ENABLED	true	active le détecteur d'anomalies
RULES_RESEED	false	réimporte rules.yaml au démarrage
RAW_RETENTION_HOURS	24	conservation des mesures brutes
ROLLUP_RETENTION_DAYS	7	conservation des agrégats par minute
ALERT_RETENTION_DAYS	30	conservation des alertes résolues
PROMETHEUS_TARGETS	—	URL d'exportateurs Prometheus à interroger
PROMETHEUS_METRICS	—	métriques Prometheus à conserver

Variable	Défaut	Rôle
`TZ`	`UTC`	fuseau horaire des journaux

ANNEXE E

Liste des captures d'écran

Les captures sont à placer dans `figures/captures/` sous le nom exact indiqué. Une capture absente est remplacée par un cadre rouge qui rappelle son nom.

Tab. E.1 : Captures d'écran à fournir

Fichier	Figure	Contenu
`cap-01-organigramme.png`	1.1	organigramme de l'entreprise
`cap-02-login.png`	4.3	page de connexion
`cap-03-premier-mot-de-passe.png`	4.3	changement obligatoire du mot de passe
`cap-04-vue-ensemble-normal.png`	4.4	vue d'ensemble sans incident
`cap-05-vue-ensemble-incident.png`	4.5	vue d'ensemble avec un incident ouvert
`cap-06-detail-pourquoi.png`	4.6	panneau d'incident, onglet Pourquoi
`cap-07-detail-chronologie.png`	4.7	onglet chronologie
`cap-08-detail-processus.png`	4.8	onglet processus
`cap-09-detail-postmortem.png`	4.9	onglet post-mortem
`cap-10-courbe-anomalie.png`	4.10	courbe avec bande attendue et point anormal
`cap-11-regles.png`	4.11	page des règles
`cap-12-parametres.png`	4.12	page des paramètres
`cap-13-journaux-conteneur.png`	4.13	journaux d'un conteneur
`cap-14-theme-clair.png`	4.14	interface en thème clair
`cap-15-mobile.png`	4.14	interface sur téléphone
`cap-16-slack-declenchement.png`	4.15	message Slack de déclenchement
`cap-17-slack-resolution.png`	4.15	message Slack de résolution
`cap-18-sen-run.png`	4.1	terminal : `sen run` jusqu'à Ready
`cap-19-sen-status.png`	4.2	terminal : `sen status`
`cap-20-ci-vert.png`	5.4	GitHub Actions, pipeline réussi
`cap-21-trivy-rouge.png`	5.5	GitHub Actions, étape Trivy en échec

Fichier	Figure	Contenu
`cap-22-npm-audit.png`	5.6	sortie de `npm audit`
`cap-23-dependabot.png`	5.7	demandes de fusion Dependabot
`cap-24-pytest.png`	5.8	terminal : `pytest`
`cap-25-npm-test.png`	5.8	terminal : `npm test`
`cap-26-docker-ps.png`	5.1	terminal : `docker ps`
`cap-27-instance-cloud.png`	5.10	console du fournisseur, instance
`cap-28-pare-feu.png`	5.11	règles de pare-feu de l'instance
`cap-29-dashboard-serveur.png`	5.12	dashboard du serveur distant
`cap-30-curl-401.png`	5.2a	terminal : `curl` sans jeton, 401
`cap-31-proxy-403.png`	5.2b	terminal : appel interdit au proxy, 403